

Лабораторная работа №2
Вариант №160401

Выполнил:
Решетников С.Е.

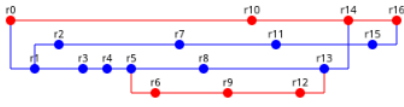
Проверила:
Наумова Н.А.

Дата сдачи: 1 апреля
Санкт-Петербург, 2026

1. Текст задания

Лабораторная работа #2

Вариант



Сконфигурировать в своём домашнем каталоге репозитории svn и git и загрузить в них начальную ревизию файлов с исходными кодами (в соответствии с выданным вариантом).

Воспроизвести последовательность команд для систем контроля версий svn и git, осуществляющих операции над исходным кодом, приведённые на блок-схеме.

При составлении последовательности команд необходимо учитывать следующие условия:

- Цвет элементов схемы указывает на пользователя, совершившего действие (красный - первый, синий - второй).
- Цифры над узлами - номер ревизии. Ревизии создаются последовательно.
- Необходимо разрешать конфликты между версиями, если они возникают.

Отчёт по работе должен содержать:

1. Задание и блок-схему в соответствии с вариантом.
2. Список команд, использованных при создании и конфигурации репозитория в домашнем каталоге пользователя.
3. Номера ревизий и соответствующие им последовательности команд с комментариями (для svn и git).
4. Выводы по работе.

2. Исходный код

2.1. Bash-скрипт

```
#!/bin/bash

usage() {
    cat << EOF
Usage: lab.bash [options] <path> <vcs>

Arguments:
    path                - Path to repo directory
    vcs                 - Type of VCS: git or svn

Options:
    -c, --commits PATH  - Commits directory. 'commits' by default
    -f, --flow PATH     - Path to repository structure description. By default searching in '[commits]/flow'
    --commits-prefix PREFIX - Prefix of commits archives: [prefix][number].zip. 'commit' by default
    --step-mode         - Step-by-step mode
    --no-history        - Skip showing history at the end of exec
    -h, --help          - Show this help message

Examples:
    $0 [repo_path] git          # run using git as VCS
    $0 [repo_path] [vcs] -c /tmp/lab/git # With custom commits dir (/tmp/lab/git/)
EOF
    exit 1
}

cd "$(dirname "$0")"

# Constants
REPO_DIR=""
COMMIT_DIR="commits"
FLOW_PATH=""
COMMIT_ARCHIVE_PREFIX="commit"
IS_STEP_BY_STEP=false
SHOW_GRAPH=true

# Colors
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m'

# Parsing args
while [ $# -gt 0 ]; do
    case "$1" in
        -h|--help)
            usage
            ;;
        -c|--commits)
    
```

```

[ $# -lt 2 ] && { echo -e "${RED}Error: commits (-c or --commits) requires an argument${NC}"; usage; }
COMMITTS_DIR="$2"
echo -e "${GREEN}Using $COMMITTS_DIR as commits dir${NC}"
shift 2
;;
-f|--flow)
[ $# -lt 2 ] && { echo -e "${RED}Error: flow (-f or --flow) requires an argument${NC}"; usage; }
FLOW_PATH="$2"
echo -e "${GREEN}Using $FLOW_PATH as flow description file${NC}"
shift 2
;;
--commits-prefix)
[ $# -lt 2 ] && { echo -e "${RED}Error: --commits-prefix requires an argument${NC}"; usage; }
COMMIT_ARCHIVE_PREFIX="$2"
echo -e "${GREEN}Using $COMMIT_ARCHIVE_PREFIX as commits prefix${NC}"
shift 2
;;
--step-mode)
IS_STEP_BY_STEP=true
echo -e "${YELLOW}Step-by-step mode enabled${NC}"
shift
;;
--no-history)
SHOW_GRAPH=false
echo "${YELLOW}History showing disabled${NC}"
shift
;;
-*)
echo -e "${RED}Unknown option: $1${NC}"
usage
;;
*)
if [ -z "$REPO_DIR" ]; then
    REPO_DIR="$1"
    echo -e "${GREEN}Repo path: $REPO_DIR${NC}"
elif [ -z "$VCS_TYPE" ]; then
    VCS_TYPE="$1"
    if [ [ "$VCS_TYPE" != "git" && "$VCS_TYPE" != "svn" ] ]; then
        echo "${RED}Error: vcs must be 'git' or 'svn'${NC}"
        usage
    fi
    echo -e "${GREEN}VCS type: $VCS_TYPE${NC}"
else
    echo "${RED}Error: Too many arguments: $1${NC}"
    usage
fi
shift
;;
esac
done

if [ -z "$FLOW_PATH" ]; then
    FLOW_PATH="$COMMITTS_DIR/flow"
fi
[ ! -f "$FLOW_PATH" ] && { echo "Flow file not found"; exit 1; }

[ -z "$REPO_DIR" ] && { echo "${RED}Error: Missing path argument${NC}"; usage; }
[ -z "$VCS_TYPE" ] && { echo "${RED}Error: Missing vcs argument${NC}"; usage; }

# STEPS

prepare_dir() {
    rm -rf "$REPO_DIR"
    mkdir -p "$REPO_DIR"
}

# GIT

init_git() {
    git init

    #git config --local merge.tool vimdiff
    #git config --local merge.conflictstyle diff3
    git config --local merge.tool vscode
    git config --local mergetool.keepBackup false
    git config --local mergetool.vscode.cmd "code --new-window --disable-extensions --wait \${MERGED}"
    git config --local mergetool.prompt false
}

apply_commit_git() {
    local id=$1
    local author=$2

    unzip -o "$COMMITTS_DIR/${COMMIT_ARCHIVE_PREFIX}${id}.zip" -d .
    git add .

    if [ "$author" = "blue" ]; then
        git commit --author="blue <blue@example.com>" -m "r$id"
    fi
}

```

```

elif [ "$author" = "red" ]; then
    git commit --author="red <red@example.com>" -m "r$id"
fi

echo "--- commit $id ($author)"
}

do_merge_git() {
    local branch=$1

    git merge --no-commit "$branch"
    git mergetool
    git add .
}

checkout_git() {
    local branch=$1
    git checkout "$branch"
}

new_branch_git() {
    local branch=$1
    git checkout -b "$branch"
}

graph_git() {
    git log --graph --abbrev-commit --decorate --all \
        --format=format:'%C(bold blue)%h%C(reset) - %C(bold green)(%ar)%C(reset) %C(white)%s%C(reset) %C(dim white)-%an%C(reset)%C(auto)%d%C(reset)'
}

# SVN

init_svn() {
    svnadmin create "$REPO_DIR"
    REPO_URL="file://$(pwd)/$REPO_DIR"

    svn mkdir -m "project structure" "$REPO_URL/trunk" "$REPO_URL/branches"
    svn checkout "$REPO_URL/trunk" wc

    cd wc
}

apply_commit_svn() {
    local id=$1 author=$2
    unzip -o "$COMMIT_DIR/${COMMIT_ARCHIVE_PREFIX}${id}.zip" -d .
    svn add * --force
    svn commit -m "Revision $id" --username="$author"
    echo "--- SVN commit $id ($author)"
}

do_merge_svn() {
    local branch=$1
    svn update
    svn merge "$REPO_URL/branches/$branch"
    svn add * --force
    echo "--- SVN merge from $branch"
}

checkout_svn() {
    local branch=$1
    svn update
    svn switch "$REPO_URL/branches/$branch"
}

new_branch_svn() {
    local branch=$1
    svn update
    svn copy "$REPO_URL/trunk" "$REPO_URL/branches/$branch" -m "Creating branch $branch"
    svn switch "$REPO_URL/branches/$branch"
}

graph_svn() {
    svn log -v
}

run_flow() {
    local init_func=$1
    local commit_func=$2
    local merge_func=$3
    local checkout_func=$4
    local new_branch_func=$5
    local graph_func=$6

    while IFS='|' read -r cmd arg1 arg2 <&3; do
        [[ "$cmd" =~ ^#.*$ || -z "$cmd" ]] && continue

```

```

case "$cmd" in
init)
    prepare_dir
    cd "$REPO_DIR"
    $init_func
    ;;

branch)
    $new_branch_func "$arg1"
    echo "--- branch $arg1"
    ;;

checkout)
    $checkout_func "$arg1"
    echo "--- checkout $arg1"
    ;;

commit)
    $commit_func "$arg1" "$arg2"
    ;;

merge)
    $merge_func "$arg1"
    ;;
end)
;;
*)
    echo "Unknown command: $cmd"
    exit 1
    ;;
esac

if [ $IS_STEP_BY_STEP = true ]; then
    echo "Press any key to continue..."
    read
fi

done 3< "$FLOW_PATH"

if [ $SHOW_GRAPH = true ]; then
    $graph_func
fi
}

if [ "$VCS_TYPE" = "git" ]; then
    run_flow init_git apply_commit_git do_merge_git checkout_git new_branch_git graph_git
elif [ "$VCS_TYPE" = "svn" ]; then
    run_flow init_svn apply_commit_svn do_merge_svn checkout_svn new_branch_svn graph_svn
fi

```

2.2. Файл конфигурации

```

init

branch|b0
commit|0|red

branch|b1
commit|1|blue

branch|b2
commit|2|blue

checkout|b1
commit|3|blue
commit|4|blue
commit|5|blue

branch|b3
commit|6|red

checkout|b2
commit|7|blue

checkout|b1
commit|8|blue

checkout|b3
commit|9|red

checkout|b0
commit|10|red

```

```
checkout|b2  
commit|11|blue
```

```
checkout|b3  
commit|12|red
```

```
checkout|b1  
merge|b3  
commit|13|blue
```

```
checkout|b0  
merge|b1  
commit|14|red
```

```
checkout|b2  
commit|15|blue
```

```
checkout|b0  
merge|b2  
commit|16|red  
end
```

3. Вывод

В ходе выполнения лабораторной работы были изучены системы контроля версий Git и Subversion. Была смоделирована работа нескольких пользователей в различных ветках, включая создание веток, выполнение последовательных коммитов, слияние изменений и разрешение возникающих конфликтов. Выполнение работы помогло мне лучше понять принципы работы с системами контроля версий.